

Module 8 Structured Note: Finding Duplicates Using Bitwise Operations

1. What this module is about

Module 8 teaches how to find duplicate letters in a string using **bitwise operations**.

In Module 7, we found duplicates using:

1. comparison with nested loops
2. a hash table/counting array like `int H[26]`

In Module 8, we use a more compact method:

```
long int H = 0;
```

Instead of using 26 integer array slots, we use **bits inside one integer**.

The main idea is:

Use one bit to remember whether each letter has already appeared.

Example:

```
a -> bit 0
b -> bit 1
c -> bit 2
...
z -> bit 25
```

So if the bit for `i` is already ON and we see `i` again, then `i` is a duplicate.

2. Required ideas from previous modules

Before Module 8, remember these ideas.

C-style strings end with `\0`

A C-style string is a character array that ends with the null terminator:

```
'\0'
```

Example:

```
char A[] = "finding";
```

Memory view:

```
Index: 0 1 2 3 4 5 6 7  
Value: f i n d i n g \0
```

The visible string is:

```
finding
```

The `\0` is not part of the word. It only tells C where the string ends.

Most string loops use this pattern:

```
for (i = 0; A[i] != '\0'; i++) {  
    // process A[i]  
}
```

Meaning:

```
Start at index 0.  
Check one character at a time.  
Stop when \0 is found.
```

Lowercase letters map to numbers

Lowercase letters have ASCII values:

```
a = 97  
b = 98  
c = 99  
...  
z = 122
```

To map a lowercase letter to a position from 0 to 25, we subtract 'a':

```
position = A[i] - 'a';
```

Examples:

```
'a' - 'a' = 0  
'b' - 'a' = 1  
'f' - 'a' = 5  
'i' - 'a' = 8  
'n' - 'a' = 13  
'z' - 'a' = 25
```

This idea was also used in Module 7 with `H[26]`.

3. Quick review: duplicates

A duplicate character is a character that appears more than once.

Example:

```
char A[] = "finding";
```

The string is:

```
f i n d i n g
```

Counts:

Character	Count	Duplicate?
f	1	No
i	2	Yes
n	2	Yes
d	1	No
g	1	No

So the duplicate letters are:

i and n

Part A: Binary basics

4. What is a bit?

A bit is the smallest unit of memory.

A bit can store only one of two values:

0 or 1

A bit is like a tiny switch:

0 = OFF

1 = ON

In Module 8, we use bits as switches.

If a letter has not appeared before, its bit is OFF.

If a letter has appeared before, its bit is ON.

5. What is a byte?

A byte has 8 bits.

Example 8-bit layout:

128 64 32 16 8 4 2 1

Each place has a value.

Moving from right to left, the values double:

1, 2, 4, 8, 16, 32, 64, 128

6. Binary examples

Decimal 1

00000001

Only the 1 position is ON.

So the value is:

1

Decimal 2

00000010

Only the 2 position is ON.

So the value is:

2

Decimal 4

00000100

Only the 4 position is ON.

So the value is:

4

Decimal 8

00001000

Only the 8 position is ON.

So the value is:

8

Decimal 10

00001010

The 8 position and the 2 position are ON.

So:

$8 + 2 = 10$

7. Why binary matters in Module 8

In Module 8, each letter is represented by one bit position.

Example:

```
a -> bit 0  
b -> bit 1  
c -> bit 2  
d -> bit 3  
e -> bit 4  
f -> bit 5  
...
```

If we want to remember that f appeared, we turn ON bit 5.

If we want to remember that i appeared, we turn ON bit 8.

So instead of storing letter counts in an array, we store letter presence in bits.

Part B: Left shift <<

8. Meaning of left shift

The left shift operator is:

<<

It moves bits to the left.

Example:

$1 \ll 0$

Result:

00000001

Decimal value:

1

Because the 1 did not move.

9. Left shift by 1

$1 \ll 1$

Binary idea:

00000001

After shifting left by 1:

00000010

Decimal value:

2

So:

$1 \ll 1 = 2$

10. Left shift by 2

$1 \ll 2$

Binary idea:

00000001

After shifting left by 2:

00000100

Decimal value:

4

So:

$1 \ll 2 = 4$

11. Left shift as a mask creator

In Module 8, left shift is used to create a **mask**.

A mask is a value with one important bit turned ON.

Pattern:

```
X = 1 << position;
```

This means:

Turn ON only the bit at the required position.

Examples:

Letter	Position	Mask expression
a	0	<code>1 << 0</code>
b	1	<code>1 << 1</code>
f	5	<code>1 << 5</code>
i	8	<code>1 << 8</code>
n	13	<code>1 << 13</code>

Part C: Mapping letters to bit positions

12. Letter position formula

For lowercase letters, the formula is:

```
position = A[i] - 'a';
```

This gives a number from `0` to `25`.

Examples:

```
'a' - 'a' = 0  
'b' - 'a' = 1  
'c' - 'a' = 2  
'd' - 'a' = 3  
'f' - 'a' = 5  
'i' - 'a' = 8  
'n' - 'a' = 13  
'z' - 'a' = 25
```

13. Example: creating a mask for `i`

Letter:

`i`

Position:

`'i' - 'a'`

Result:

`8`

Now create the mask:

`X = 1 << 8;`

This creates a number where bit 8 is ON.

Meaning:

This mask represents the letter `i`.

14. Example: creating a mask for `n`

Letter:

`n`

Position:

`'n' - 'a'`

Result:

13

Now create the mask:

```
X = 1 << 13;
```

This creates a number where bit 13 is ON.

Meaning:

This mask represents the letter n.

Part D: AND &

15. Meaning of AND

The AND operator is:

&

AND compares bits.

It gives 1 only when both bits are 1.

Truth table:

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Simple meaning:

AND checks whether the same bit is ON in both values.

16. AND as masking

In Module 8, AND is used for checking.

X & H

Here:

X = mask for the current letter
H = all letters seen so far

So:

X & H

asks:

Is this letter already present in H?

If the answer is 0:

The letter has not appeared before.

If the answer is greater than 0:

The letter has appeared before, so it is duplicate.

17. Example of AND check

Suppose bit 8 represents i.

If i has not appeared before, bit 8 in H is OFF.

So:

X & H

returns:

0

Meaning:

i is not duplicate yet.

But if **i** already appeared, bit 8 in **H** is ON.

Then:

X & H

returns a value greater than **0**.

Meaning:

i is duplicate.

Part E: OR **|**

18. Meaning of OR

The OR operator is:

|

OR compares bits.

It gives **1** if at least one bit is **1**.

Truth table:

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Simple meaning:

OR turns a bit ON if either side has that bit ON.

19. OR as merging

In Module 8, OR is used to store a letter as seen.

`H = X | H;`

Here:

X = mask for the current letter
H = all letters seen so far

So this line means:

Add the current letter's bit into H.

It turns ON the current letter's bit without disturbing the other bits.

20. Example of OR storage

Suppose we see `f` for the first time.

Position of `f`:

5

Mask:

```
X = 1 << 5;
```

Now store it:

```
H = X | H;
```

Meaning:

```
Turn ON bit 5 in H.
```

Now `H` remembers that `f` appeared.

Part F: Variables used in the algorithm

21. Variable `H`

```
long int H = 0;
```

`H` stores all letters that have already appeared.

At the beginning:

```
H = 0
```

This means:

```
No letter has been seen yet.
```

As we scan the string, bits inside `H` turn ON.

Example:

If f appeared, bit 5 is ON.
If i appeared, bit 8 is ON.
If n appeared, bit 13 is ON.

22. Variable `X`

```
long int X = 0;
```

`X` is the mask for the current character.

For each character, we reset:

```
X = 1;
```

Then we shift it:

```
X = X << (A[i] - 'a');
```

This creates a mask for the current letter.

Example:

If A[i] is f, X becomes mask for bit 5.
If A[i] is i, X becomes mask for bit 8.
If A[i] is n, X becomes mask for bit 13.

23. Why `X = 1` is reset each time

This line is important:

```
X = 1;
```

Why?

Because every new character needs a fresh mask.

If we do not reset `X`, then previous shifts can affect the next character.

Correct pattern:

```
X = 1;
X = X << (A[i] - 'a');
```

Meaning:

```
Start with one ON bit.
Move that bit to the current letter's position.
```

Part G: Main algorithm

24. Algorithm in simple words

For each character in the string:

1. Start with `X = 1`.
2. Find the letter position using `A[i] - 'a'`.
3. Shift `X` left by that position.
4. Use `X & H` to check whether the bit is already ON.
5. If the bit is already ON, print duplicate.
6. Otherwise, use `H = X | H` to turn that bit ON.

25. Main code pattern

```
long int H = 0;
long int X = 0;

for (i = 0; A[i] != '\0'; i++) {
    X = 1;
    X = X << (A[i] - 'a');

    if ((X & H) > 0) {
        printf("%c is duplicate\n", A[i]);
    } else {
        H = X | H;
    }
}
```

```
}  
}
```

26. Full program

```
#include <stdio.h>  
  
int main() {  
    char A[] = "finding";  
  
    long int H = 0;  
    long int X = 0;  
    int i;  
  
    for (i = 0; A[i] != '\0'; i++) {  
        X = 1;  
        X = X << (A[i] - 'a');  
  
        if ((X & H) > 0) {  
            printf("%c is duplicate\n", A[i]);  
        } else {  
            H = X | H;  
        }  
    }  
  
    return 0;  
}
```

Output:

```
i is duplicate  
n is duplicate
```

Part H: Dry run for finding

27. Starting point

String:

```
f i n d i n g
```

Initial values:

```
H = 0
X = 0
```

Meaning:

```
No letter has been seen yet.
```

28. Step-by-step dry run

Step	Character	Position	Mask	X & H result	Meaning	Action
1	f	5	1 << 5	0	f not seen before	Turn ON bit 5
2	i	8	1 << 8	0	i not seen before	Turn ON bit 8
3	n	13	1 << 13	0	n not seen before	Turn ON bit 13
4	d	3	1 << 3	0	d not seen before	Turn ON bit 3
5	i	8	1 << 8	greater than 0	i already seen	Print duplicate
6	n	13	1 << 13	greater than 0	n already seen	Print duplicate
7	g	6	1 << 6	0	g not seen before	Turn ON bit 6

Final output:

```
i is duplicate
n is duplicate
```

29. Detailed explanation of the first `i`

Character:

```
i
```

Position:

```
'i' - 'a' = 8
```

Mask:

```
X = 1 << 8;
```

Now check:

```
X & H
```

At this point, `i` has not appeared before.

So the result is:

```
0
```

Meaning:

```
i is not duplicate.
```

Then we store it:

```
H = X | H;
```

Now `H` remembers that `i` appeared.

30. Detailed explanation of the second `i`

When the second `i` appears:

```
'i' - 'a' = 8
```

Again:

```
X = 1 << 8;
```

Now check:

```
X & H
```

This time, bit 8 is already ON in `H`.

So the result is greater than `0`.

Meaning:

```
i has appeared before.
```

So we print:

```
i is duplicate
```

31. Detailed explanation of the second `n`

When the second `n` appears:

```
'n' - 'a' = 13
```

Mask:

```
X = 1 << 13;
```

Check:

```
X & H
```

Bit 13 is already ON in `H`.

So the result is greater than `0`.

Meaning:

```
n has appeared before.
```

So we print:

```
n is duplicate
```

Part I: Mental model

32. Think of **H** as switches

Imagine **H** as a row of switches:

```
a b c d e f g h i j k l m n ... z
0 0 0 0 0 0 0 0 0 0 0 0 0 0 ... 0
```

At the start, all switches are OFF.

When we see **f**, turn ON the **f** switch:

```
a b c d e f g h i j k l m n ... z
0 0 0 0 0 1 0 0 0 0 0 0 0 0 ... 0
```

When we see **i**, turn ON the **i** switch:

```
a b c d e f g h i j k l m n ... z
0 0 0 0 0 1 0 0 1 0 0 0 0 0 ... 0
```

When we see **i** again, the **i** switch is already ON.

So **i** is duplicate.

Main rule:

```
OFF = first time seeing the letter  
ON  = letter was already seen before
```

Part J: Why this works for 26 lowercase letters

33. One integer can store many bits

A common integer has enough bits to store the 26 lowercase letters.

Lowercase English has 26 letters:

```
a to z
```

So we need 26 bit positions:

```
bit 0 to bit 25
```

That is why one integer can act like a compact set of letters.

The source code uses:

```
long int H = 0;
```

This gives enough space for the bit positions needed in this beginner example.

34. Why this is memory efficient

Module 7 hash table method used:

```
int H[26] = {0};
```

That uses 26 integer slots.

Module 8 uses:

```
long int H = 0;
```

That uses one integer variable.

So Module 8 uses less extra memory.

Part K: Comparison with Module 7

35. Module 7 hash table vs Module 8 bitwise method

Feature	Module 7 hash table	Module 8 bitwise method
Storage	<code>int H[26]</code>	one integer <code>H</code>
Main idea	Count each letter	Store whether each letter appeared
Can detect duplicates?	Yes	Yes
Can count exact frequency?	Yes	No
Extra space	Fixed 26 integers	One integer
Time complexity	<code>O(N)</code>	<code>O(N)</code>
Best use	When counts are needed	When only duplicate detection is needed

36. Important difference: counting vs presence

The bitwise method tracks only presence.

It knows:

Has this letter appeared before?

It does not naturally know:

How many times did this letter appear?

Example:

```
banana
```

Bitwise method can detect repeated letters:

```
a is duplicate  
n is duplicate  
a is duplicate
```

But it does not directly say:

```
a appears 3 times  
n appears 2 times
```

For exact counts, use the Module 7 hash table method.

Part L: Safer version for lowercase letters only

37. Problem with the basic code

This line assumes the character is lowercase:

```
X = X << (A[i] - 'a');
```

It works for:

```
finding  
banana  
programming
```

But it is not safe for:

```
Finding123  
hello!  
abc def
```

Why?

Because uppercase letters, digits, spaces, and symbols do not map safely to positions `0` to `25`.

Example:

```
'F' - 'a'
```

This does not give a valid lowercase index.

38. Safer lowercase-only code

```
#include <stdio.h>

int main() {
    char A[] = "finding";

    long int H = 0;
    long int X = 0;
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'a' && A[i] <= 'z') {
            X = 1;
            X = X << (A[i] - 'a');

            if ((X & H) > 0) {
                printf("%c is duplicate\n", A[i]);
            } else {
                H = X | H;
            }
        }
    }

    return 0;
}
```

This version ignores characters that are not lowercase letters.

39. Handling uppercase letters

If you want to handle uppercase letters like lowercase letters, convert them first.

Example:

Finding

Convert to:

finding

Conversion pattern:

```
if (A[i] >= 'A' && A[i] <= 'Z') {  
    A[i] = A[i] + 32;  
}
```

Then use the bitwise duplicate logic.

Part M: Complexity analysis

40. Time complexity

The algorithm scans the string once:

```
for (i = 0; A[i] != '\0'; i++)
```

If the string has characters, the loop runs times.

So time complexity is:

$O(N)$

41. Space complexity

The algorithm uses only a few variables:

```
long int H;  
long int X;  
int i;
```

It does not create another array.

So extra space complexity is:

$O(1)$

This means constant extra space.

The amount of extra memory does not grow when the string becomes longer.

42. Complexity table

Operation	Time complexity	Extra space
Bitwise duplicate detection	$O(N)$	$O(1)$
Create mask with left shift	$O(1)$	$O(1)$
Check duplicate with AND	$O(1)$	$O(1)$
Store seen letter with OR	$O(1)$	$O(1)$

Part N: Common mistakes

43. Mistake 1: Forgetting to reset ☒

Wrong:

```
X = X << (A[i] - 'a');
```

without resetting ☒ each time.

Correct:

```
X = 1;  
X = X << (A[i] - 'a');
```

Reason:

Each character needs a fresh mask.

44. Mistake 2: Using OR before checking duplicate

Wrong order:

```
H = X | H;  
if ((X & H) > 0) {  
    printf("duplicate");  
}
```

Problem:

After OR, the bit is definitely ON, so the check becomes meaningless.

Correct order:

```
if ((X & H) > 0) {  
    printf("duplicate");  
} else {  
    H = X | H;  
}
```

Meaning:

First check whether the letter was already seen.
Only if it was not seen, store it.

45. Mistake 3: Confusing & and |

Use AND & to check:

X & H

Use OR | to store:

```
H = X | H;
```

Simple memory trick:

```
& asks: is it already there?  
| adds: put it there.
```

46. Mistake 4: Thinking the bitwise method counts frequency

The bitwise method does not count exact frequency.

It only tracks whether a letter has appeared before.

For example:

```
banana
```

The bitwise method detects repeated appearances.

But it does not naturally store:

```
a = 3  
n = 2  
b = 1
```

Use `H[26]` if exact counts are required.

47. Mistake 5: Not checking lowercase range

Risky:

```
X = X << (A[i] - 'a');
```

This assumes `A[i]` is from `'a'` to `'z'`.

Safer:

```
if (A[i] >= 'a' && A[i] <= 'z') {  
    X = 1;  
    X = X << (A[i] - 'a');  
}
```

Reason:

Uppercase letters, digits, spaces, and symbols may produce invalid shift positions.

48. Mistake 6: Forgetting to stop at `\0`

Wrong:

```
for (i = 0; i < 100; i++)
```

Better:

```
for (i = 0; A[i] != '\0'; i++)
```

Reason:

The real end of a C-style string is marked by `\0`.

Part O: Main code patterns to remember

49. Pattern 1: Scan a string

```
for (i = 0; A[i] != '\0'; i++) {  
    // process A[i]  
}
```

50. Pattern 2: Convert lowercase letter to position

```
A[i] - 'a'
```

Examples:

```
a -> 0  
b -> 1  
f -> 5  
i -> 8  
n -> 13  
z -> 25
```

51. Pattern 3: Create a mask

```
X = 1;  
X = X << (A[i] - 'a');
```

Meaning:

Create a value with only the current letter's bit ON.

52. Pattern 4: Check whether the letter is duplicate

```
if ((X & H) > 0) {  
    printf("%c is duplicate\n", A[i]);  
}
```

Meaning:

If this letter's bit is already ON in H, it is duplicate.

53. Pattern 5: Store the letter as seen

```
H = X | H;
```

Meaning:

```
Turn ON the current letter's bit in H.
```

54. Pattern 6: Full bitwise duplicate detection

```
long int H = 0;
long int X = 0;

for (i = 0; A[i] != '\0'; i++) {
    X = 1;
    X = X << (A[i] - 'a');

    if ((X & H) > 0) {
        printf("%c is duplicate\n", A[i]);
    } else {
        H = X | H;
    }
}
```

Part P: Practice questions

55. Practice 1: Write the AND truth table

Fill this table from memory:

A	B	A & B
0	0	?
0	1	?
1	0	?
1	1	?

Answer:

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

56. Practice 2: Write the OR truth table

Fill this table from memory:

A	B	A B
0	0	?
0	1	?
1	0	?
1	1	?

Answer:

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

57. Practice 3: Find bit positions

Find the bit position for each letter:

a, b, f, i, n

Answers:

```
a -> 0
b -> 1
f -> 5
i -> 8
n -> 13
```

58. Practice 4: Trace first four letters of finding

String:

f i n d

Trace:

Character	Position	Action
f	5	Turn ON bit 5
i	8	Turn ON bit 8
n	13	Turn ON bit 13
d	3	Turn ON bit 3

After these four letters, H remembers:

f, i, n, d

59. Practice 5: Explain why this method cannot count exact frequency

Question:

Why can the bitwise method detect duplicates but not count exact repeats?

Answer:

Because each letter gets only one bit.
A bit can only be OFF or ON.
It cannot store 2, 3, 4, or more.

So the bit can say:

The letter appeared.

But it cannot say:

The letter appeared 3 times.

Part Q: Final summary

60. Key points to remember

1. Module 8 finds duplicates using bitwise operations.
 2. A bit can store only `0` or `1`.
 3. A byte has 8 bits.
 4. Binary place values double as we move left: `1, 2, 4, 8, 16, 32, 64, 128`.
 5. Left shift `<<` moves bits to the left.
 6. `1 << position` creates a mask for one letter.
 7. Lowercase letters map to positions using `A[i] - 'a'`.
 8. `a` maps to bit 0, `b` maps to bit 1, and `z` maps to bit 25.
 9. `H` stores all letters seen so far.
 10. `X` stores the mask for the current letter.
 11. AND `&` is used to check whether a letter was already seen.
 12. OR `|` is used to store a letter as seen.
 13. If `(X & H) > 0`, the letter is duplicate.
 14. If `(X & H) == 0`, the letter is appearing for the first time.
 15. `H = X | H` turns ON the current letter's bit.
 16. The algorithm scans the string once.
 17. Time complexity is `O(N)`.
 18. Extra space complexity is `O(1)`.
 19. The basic method assumes lowercase letters from `a` to `z`.
 20. This method detects duplicates but does not count exact frequency.
-

61. Final mental model

Think of the integer `H` as 26 tiny switches:

```
a b c d e f g h i j k l m n ... z
0 0 0 0 0 0 0 0 0 0 0 0 0 0 ... 0
```

Each switch belongs to one letter.

When a letter appears for the first time, turn its switch ON.

When the same letter appears again, its switch is already ON.

So it is duplicate.

Main idea:

```
OFF means first time.
ON means already seen.
Already seen means duplicate.
```

62. Module 8 checkpoint

You are ready for Module 9 if you can do these without help:

1. Explain what a bit is.
2. Explain what left shift `<<` does.
3. Write the AND truth table.
4. Write the OR truth table.
5. Map letters like `a`, `f`, `i`, and `n` to bit positions.
6. Explain what `H` stores.
7. Explain what `X` stores.
8. Explain why `X = 1` must be reset each time.
9. Explain how `(X & H) > 0` detects a duplicate.
10. Explain how `H = X | H` stores a seen letter.
11. Dry run the word `finding`.
12. State the time complexity as `O(N)`.
13. State the extra space complexity as `O(1)`.
14. Explain why this method detects duplicates but does not count exact frequencies.

Final takeaway:

Module 7 used an array to count letters.
Module 8 uses bits to remember letters.
Both can find duplicates, but the bitwise method is more compact.